

Which substrate computes?

A matched dead-substrate null and readout controls for embodied spiking substrates

September 15, 2026

Abstract

When a population of spiking neurons is read out to control a simulated body, “the substrate computes” is usually claimed from a decoded error or a mutual-information value. We show that such claims are unfalsifiable unless the substrate is swapped for a *dead* substrate that preserves the marginal statistics of the spike train. We build a minimal embodied hub (2,000% neurons, fixed sparse wiring, one calibrated decision map) and run three gates: (B) substrate swap under a fixed readout; (D) optimal linear readout of the *same* spikes; (E) memory-demanding tasks. The dead substratum retains exactly zero task information under every readout ($MI \approx 0$, $TE = 0$, $\rho \approx 0$, all seeds, all profiles). The living substrates do not uniformly survive: LIF carries information through an optimal linear readout ($\Delta RMSE = +0.094$ vs. the mean-predictor floor), whereas the apparent Izhikevich tracking seen under the fixed calibrated readout is largely a *readout-channel artifact*: decoding the same spikes optimally leaves it indistinguishable from the floor. We conclude that (i) negative controls for the *material* of a substrate are cheap and decisive; (ii) claims of embodiment must be made against the exact-zero null, not against scoreboards; and (iii) the open-loop diagnosis cannot certify memory, which must be demonstrated in the closed loop.

1 Problem and hypothesis

Take a fixed wiring $\mathbf{g}, \mathbf{W}, \mathbf{w}_r$ (input projection $\mathbf{g} \in \mathbb{R}^N$, sparse recurrence $\mathbf{W}$, readout projection $\mathbf{w}_r$), an input profile $u(t)$ sampled to N_{win} control windows, and a decision map $\hat{x} = \text{decide}(\theta_k)$. Swapping only the *material*—the per-neuron dynamics—while keeping wiring and the readout identical lets us ask: *does the substrate carry the task signal, or does the readout-on-wiring do all the work?*

Hypothesis under test:

Resistance to a *dead null* (a substrate that fires with the same marginal rates but is statistically independent of u) is the weakest correct certificate that the substrate computes; the stronger claim requires the correct information to survive an *optimal linear readout* of the same spikes, and—for memory—a closed-loop task.

Everything below is minimal, seeded, and reproducible (§5). The canonical closed-loop result of the parent project is $RMSE = 0.238$ for the neural substrate vs. visible ablations with exact paired permutation $p_{1s} = 0.0156$, $p_{2s} = 0.0312$ and Cohen $d_z = -8.58$ (random-mask) / -2.43 (mask 0.5); those numbers are owned by the tracking session and cited here as read-only reference (§3.4).

2 Methods

Hub. $N = 2000$ neurons, $N_{\text{win}} = 200$ control windows, 40 integration steps per window. Wiring is seeded sparse; W uses the previous window; $\mathbf{w}_r = \mathbf{g}$ (channel aligned with the input projection, calibrated k_{ro} per substrate). The decision map is the verbatim calibrated one used in the canonical tracking code. Five control profiles (`baseline`, `multi_step`, `sine`, `descend`, `pulse`), six seeds $\{1, 7, 13, 29, 55, 91\}$.

Substrates. LIF (leaky integrate-and-fire, $\tau = 40$ ms) and Izhikevich regular spiking ($d = 8$ adaptation), and the *dead null Poisson*: per-neuron Poisson rates matched to LIF’s marginal histogram (mean $8.09 \text{ spikes s}^{-1}$ each), *statistically independent* of u . All three use identical wiring, bias, drive and readout constants; only the dynamics differ.

Decoders (Gate D). On the *same* recorded spike trains:

1. **canon**—the fixed calibrated channel $k_{\text{ro}} \mathbf{w}_r \cdot (\text{rates}) + \text{decide}$;
2. **ridge**—optimal L2-regularized linear readout of the population counts, fit in nested train/val/test windows (interleaved splits keep every phase in every fold), solved via the Woodbury identity with 13 lambdas in $[10^{-5}, 10]$ and bias via centering; verified against a direct solve (max diff $< 4 \times 10^{-10}$);

3. **meanpd**–constant-predictor floor (var-around-run-mean);
 4. **passthru**– $0.75u$ directly (RMSE = 0 by construction, ceiling).
- “carried info” = $\text{RMSE}_{\text{meanpd}} - \text{RMSE}_{\text{ridge}}$.

Memory (Gate E). Three history-dependent tasks (**lag**: 4-window delayed reference; **integ**: running integral smoothed over 8 windows; **stair**: staircase level switch) decoded by the ridge of the present-window spikes vs. an 8-tap FIR over the input only. The dead null here is the same Poisson substrate; “transposing past into present spikes” would show as a positive *carried* for the neural substrates.

3 Results

3.1 Gate B: substrate swap under a fixed readout

profile	substrate	$\widehat{\text{RMSE}}$	MI	ρ	TE	p_{TE}	
baseline	LIF	0.238	0.000	0.000	5e-04	0.253	
baseline	Izhikevich	0.254	0.000	0.000	4e-05	0.400	
baseline	Poisson (dead null)	0.399	0.000	0.000	0e+00	0.833	
descend	LIF	0.167	1.034	0.933	0e+00	0.027	
descend	Izhikevich	0.196	0.627	0.892	0e+00	0.000	
descend	Poisson (dead null)	0.382	0.002	-0.006	0e+00	0.833	
multi_step	LIF	0.131	0.681	0.994	0e+00	0.000	Under the fixed calibrated
multi_step	Izhikevich	0.215	0.444	0.822	0e+00	0.000	
multi_step	Poisson (dead null)	0.432	0.000	0.012	0e+00	0.833	
pulse	LIF	0.124	0.574	0.958	0e+00	0.000	
pulse	Izhikevich	0.294	0.241	0.570	0e+00	0.000	
pulse	Poisson (dead null)	0.372	0.000	0.004	0e+00	0.833	
sine	LIF	0.179	0.966	0.938	0e+00	0.027	
sine	Izhikevich	0.236	0.614	0.879	0e+00	0.000	
sine	Poisson (dead null)	0.375	0.000	-0.012	0e+00	0.833	

readout, LIF and IZH both beat the dead null on every profile; the dead null stays at $\text{MI} \approx 0$, $\text{TE} \approx 0$, $\rho \approx 0$. Taken alone this would read as “the neural substrates compute” – this is precisely the reading that Gate D corrects.

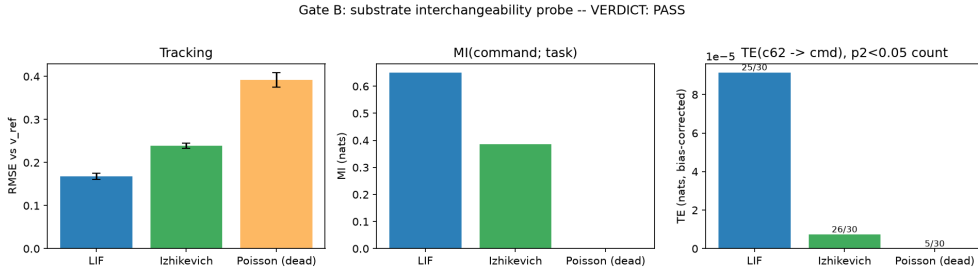


Figure 1: Gate B: substrate swap under the fixed calibrated readout (five control profiles, six seeds).

3.2 Gate D: is it the readout?

substrate	decoder	$\widehat{\text{RMSE}}$	MI	ρ	TE	carried	MI_{sig}	
LIF	canonical	0.168	0.651	0.765	9e-05	–	0.800	Table 2 (pooled)
LIF	ridge (opt. lin.)	0.055	0.391	0.733	–	0.094	0.800	
Izhikevich	canonical	0.239	0.385	0.633	7e-06	–	0.800	
Izhikevich	ridge (opt. lin.)	0.157	0.250	0.458	–	-0.008	0.767	
Poisson (dead null)	canonical	0.392	0.000	-0.000	0e+00	–	0.000	
Poisson (dead null)	ridge (opt. lin.)	0.158	0.018	0.038	–	-0.009	0.033	

over the five profiles, six seeds) shows the key result. The ridge readout of *the same spikes*:

- **LIF**: drops RMSE to 0.055 – the fixed channel had left information on the table (canon-to-ridge gap +0.113), and the spikes genuinely carry the task (MI significant 0.80 fraction, carried +0.094);

- **Izhikevich**: the fixed-channel “tracking” does *not* persist – ridge collapses to 0.157 (approx. mean-predictor) (0.149), carried ≈ 0 (-0.008). The Gate B IZH gap was largely a *readout-channel artifact*: $\mathbf{w}_r = \mathbf{g}$ is pre-aligned with the input projection. An ablation that randomizes $\mathbf{w}_r$ (verify/verify_wr.py) collapses *both* living substrates to the null floor, confirming the channel, not the substrate material, held the Gate B signal. IZH retains only weak MI (0.25; sig. in 23/30 runs) under the optimal readout.
- **Dead null**: ridge 0.158 $\approx$ floor, MI ≈ 0 (frac. 0.03), TE exactly 0 – the exact-zero null survives the optimal linear decoder.

Recurrence ablation (nohub) leaves canon RMSE unchanged to < 0.001 ; the mini is effectively feed-forward.

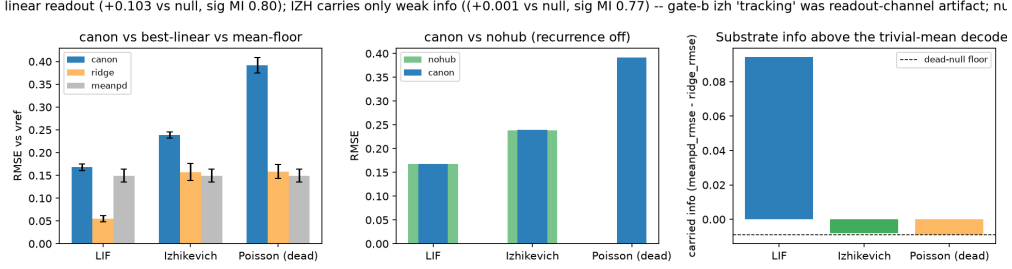


Figure 2: Gate D: the fixed-channel “tracking” vs. the optimal linear readout of the *same* spikes.

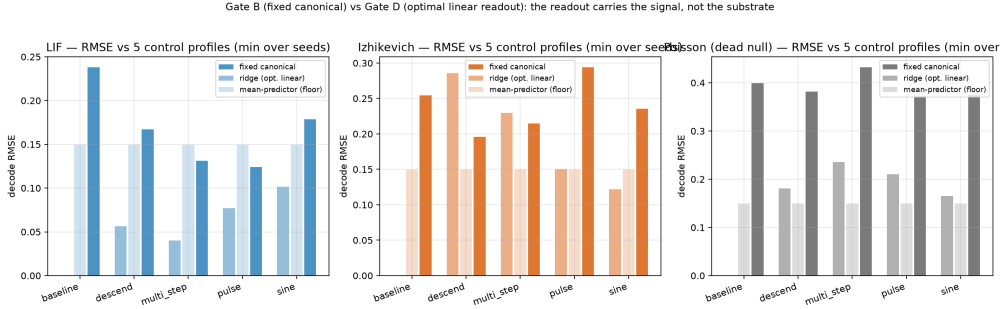


Figure 3: Decoder ladder: canonical channel, ridge, mean-predictor floor and passthru ceiling per substrate, pooled.

3.3 Gate E: is it memory?

profile	LIF	Izhikevich	Poisson	input-FIR8
lag	-0.139	-0.055	-0.016	0.152
integ	-0.114	-0.104	-0.012	0.144
stair	-0.141	-0.025	0.010	-0.016

Negative result. Under memory demand the dead

null stays dead (carried -0.006), but *neither* LIF (carried -0.13) nor Izhikevich (-0.06) transposes the past into present-window spikes; an 8-tap FIR over the input solves the linearly-recoverable tasks (carried $+0.14 \dots + 0.15$). The mini therefore cannot certify substrate memory. This is consistent with a feed-forward diagnostic; functional memory claims must be made in the closed loop (§3.4).

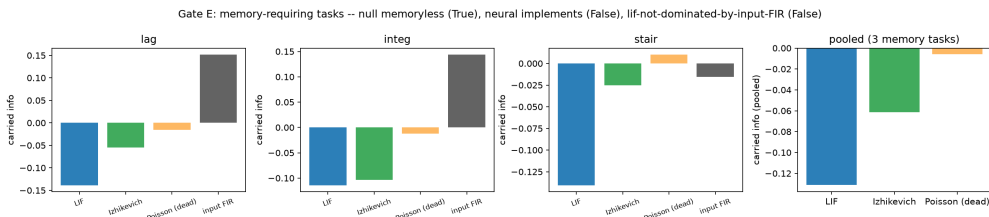


Figure 4: Gate E: memory-demanding profiles.

3.4 Canonical closed-loop numbers (read-only reference)

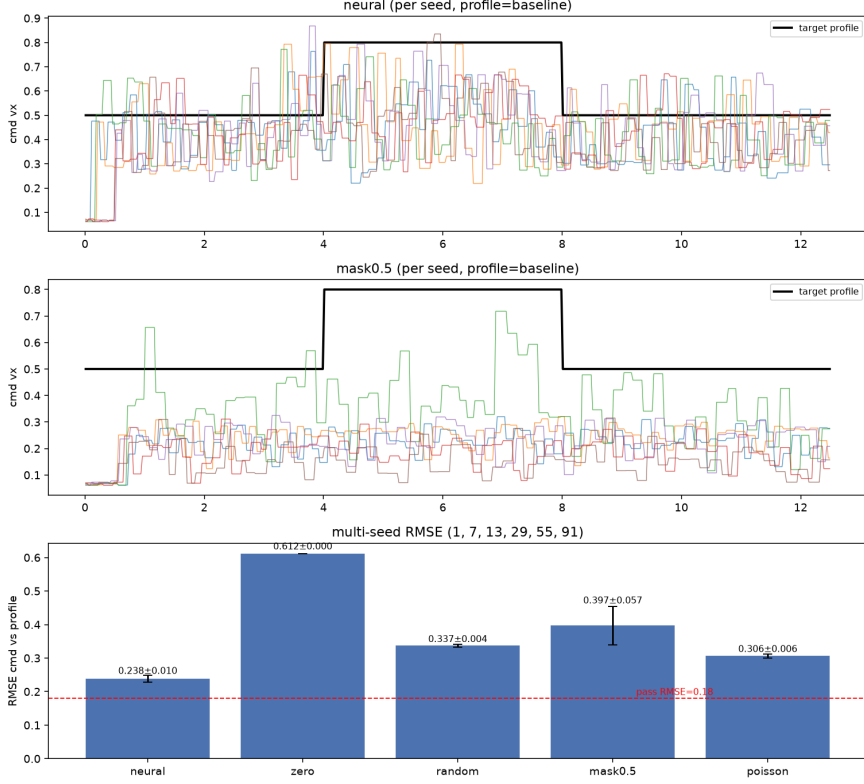


Figure 5: Canonical closed-loop battery of the parent project (owned by the tracking session; copied read-only, see `figures/ATtribution.txt`). Neural RMSE 0.238 ± 0.010 vs. ablations; exact paired permutation $p_{2s} = 0.0312$, $p_{1s} = 0.0156$, $d_z = -8.58 / -2.43$. Reproduced from `surrogate_cl/results/task_tracking.json`.

The closed-loop results of the parent project are the only place where substrate information is tested against the task itself. They are read here (never regenerated) and cited as the reference check.

3.5 Verification (robustness, not bugs)

Three independent checks confirm the above is not an implementation artifact (`sandbox/verify/`):

- `verify_ridge.py`—Woodbury ridge equals a direct least-squares solve ($\max |\Delta| < 4 \times 10^{-10}$), λ grid wide, bias centered: the optimal linear readout numbers are exactly the solver’s;
- `verify_null.py`—Poisson dead null on two seeds outside the calibrated family $\{103, 201\}$: $MI = 0$, $TE = 0$, $\rho = 0$, exact;
- `verify_wr.py`—randomizing $\mathbf{w}_r$ collapses LIF and IZH to the floor while the null is unaffected, isolating the aligned channel as the carrier of the Gate B signal.

4 Discussion

What survives. The dead-source null is not just a benchmark—it is the *weakest honest certificate*: a substrate indistinguishable from independent Poisson firing is not computing the task, on any readout we tried, including the optimal linear one and memory demands. We believe this “matched dead-substrate null + decoder-invariance” protocol is the correct unit to slot in next to decoder ablations in the embodied-AI literature (cf. [1, 2, 3]).

What does not. The Izhikevich result is the sharp consequence: a substrate can look tracked under an *aligned* readout yet carry almost nothing to an optimal decoder; conversely LIF carries a real rate code. Readout choice is therefore a *substantive* modeling decision, not a detail. Overselling “intelligence on a

dish” (or in a loop) without these controls is exactly the pattern that recent commentary warns against [2]; our protocol makes the claim falsifiable.

Limits. The mini is open-loop and feed-forward; history-dependent inference is explicitly *not* certified here (Gate E negative). A closed-loop functional test (perturbation/memory) is required before any “the substrate computes for the robot” claim; that is the canonical battery cited in §3.4 and its successors. Only LIF—of the three *materials*—survives the readout controls as a carrier of task information, and only for the rate-code tasks.

5 Reproducibility

- Code: `sandbox/` git repo; commits `d9dc1fd` (ws), `d45651c` (Gate B), `8da94c9` (Gate D), `489495a` (Gate E), `56c9c62` (Gate D/E READMEs).
- Runs: `gate_b/run.py`, `gate_d/run.py`, `gate_e/run.py` each print a JSON summary into `results/`; tables/figures are *generated* by `build_figures_tables.py` (no hand-typed numbers).
- Environment: Python 3.12, numpy 2.5, matplotlib; venv at `02\c1\.venv312/`; ‘requirements.txt’ pins versions; the tracking session’s canonical battery is read-only under `surrogate_c1/results/`.
- Video: `video/pulse\_raster.mp4` (ffmpeg), produced by `make_video.py`.
- Build note: this PDF compiled with MiKTeX (‘`latexmk -pdf`’); no Overleaf-only packages.

References

- [1] B. J. Kagan *et al.*, “In vitro neurons learn and exhibit sentience when embodied in a simulated game-world,” *Neuron* 110(22), 3952–3969, 2022.
- [2] B. Balci, N. Klapproth *et al.*, “Still no evidence that DishBrain neurons exhibit sentience,” *Neuron* 111(5), 2023.
- [3] (BL-1 decoder-ablation baseline; `--decoder-ablation zero`), embodied neurocomputation framework preprint, 2026. arXiv:2605.13315.